

Modelling Magnetic Induction Using Python

Roaa A. Alghamdi 2240006489@iau.edu.sa

Renas A. Alzhrani 2240001587@iau.edu.sa

Mayar A. Alghamdi 2230006326@iau.edu.sa

Supervised by: Dr. Gahda Ameerh Ms. Shahad Alosaimi

Haneen S. Alzhrani 2240005566@iau.edu.sa

Raghad A. Albassami 2240001254@iau.edu.sa



Abstract

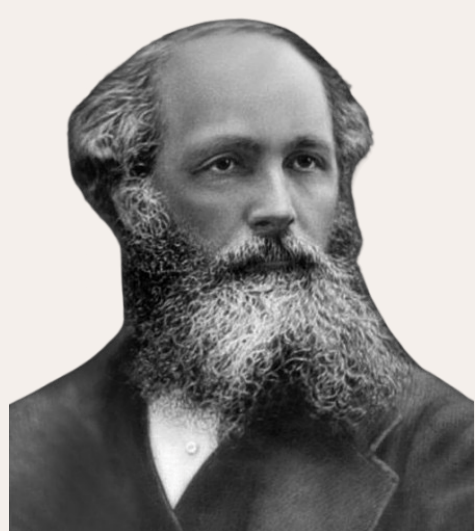
In this poster, the Magnetic Induction experiment is modeled using Python to explore a key physical phenomenon: the generation of EMF through changing magnetic fields. Experimental values were first presented using **Pandas** tables. Then, relationships between EMF, frequency, and current intensity were analyzed graphically with **Matplotlib**. Finally, error margins were estimated using a custom Python function. This modeling approach highlights the role of programming in visualizing and analyzing physics experiments efficiently.

Theoretical Background

Magnetic induction was discovered by Michael Faraday in 1831. Later Maxwell described it mathematically and it came to be known as Faraday's law of induction. [1]



Michael Faraday



James Maxwell

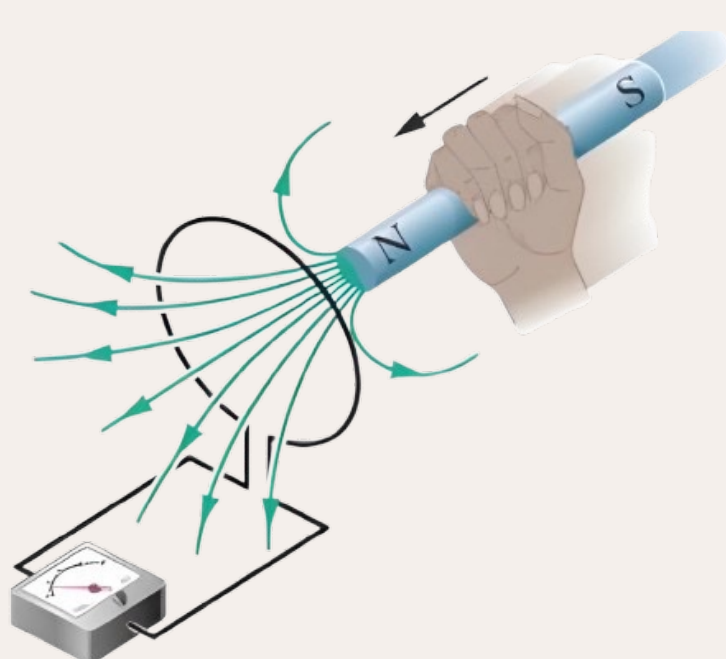


Fig.1 The magnet's motion creates a current in the loop. [3]

Magnetic induction, or electromagnetic induction, is a basic physical phenomenon where an electric current is generated in a conductor due to a changing magnetic field. Discovered by Michael Faraday in 1831, it underpins generators, transformers, and other electromechanical systems. According to Faraday's Law, the induced electromotive force (EMF) in a closed circuit is proportional to the rate of change of magnetic flux: **EMF = -dΦB/dt**. ΦB is the magnetic flux, the product of magnetic field strength (B), area (A), and the angle between them. The negative sign follows Lenz's Law, meaning the induced current opposes the flux change. [2]

Objectives

- ✦ Study the relationship between electromotive force and current intensity.
- ✦ Study the relationship between electromotive force and the current frequency.
- ✦ Simulate the generation of electromotive force (EMF) caused by changing magnetic fields.

Electric circuit

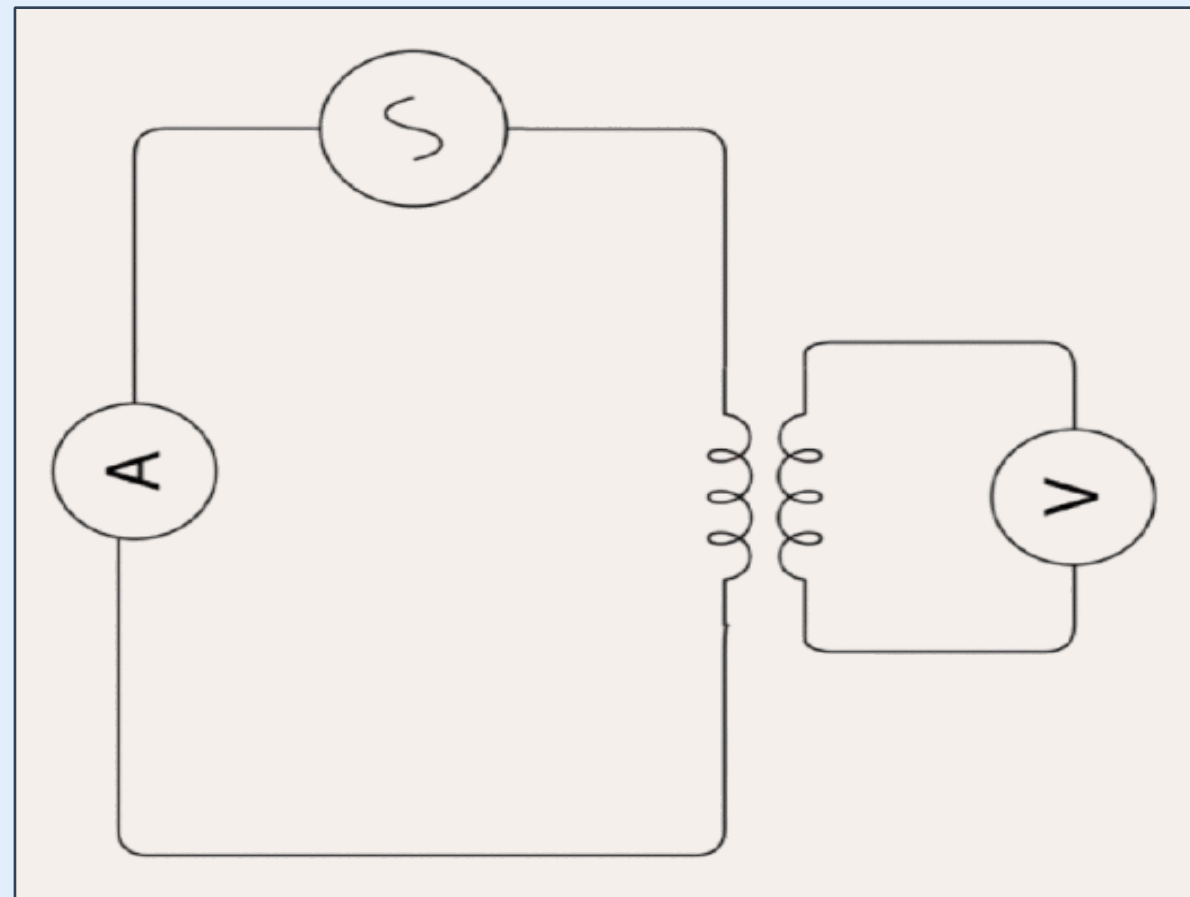


Fig.2 The electric circuit of the experiment

Experimental data

ε (mV)	f (Hz)
1.53	25
3.12	50
4.61	75
6.1	100

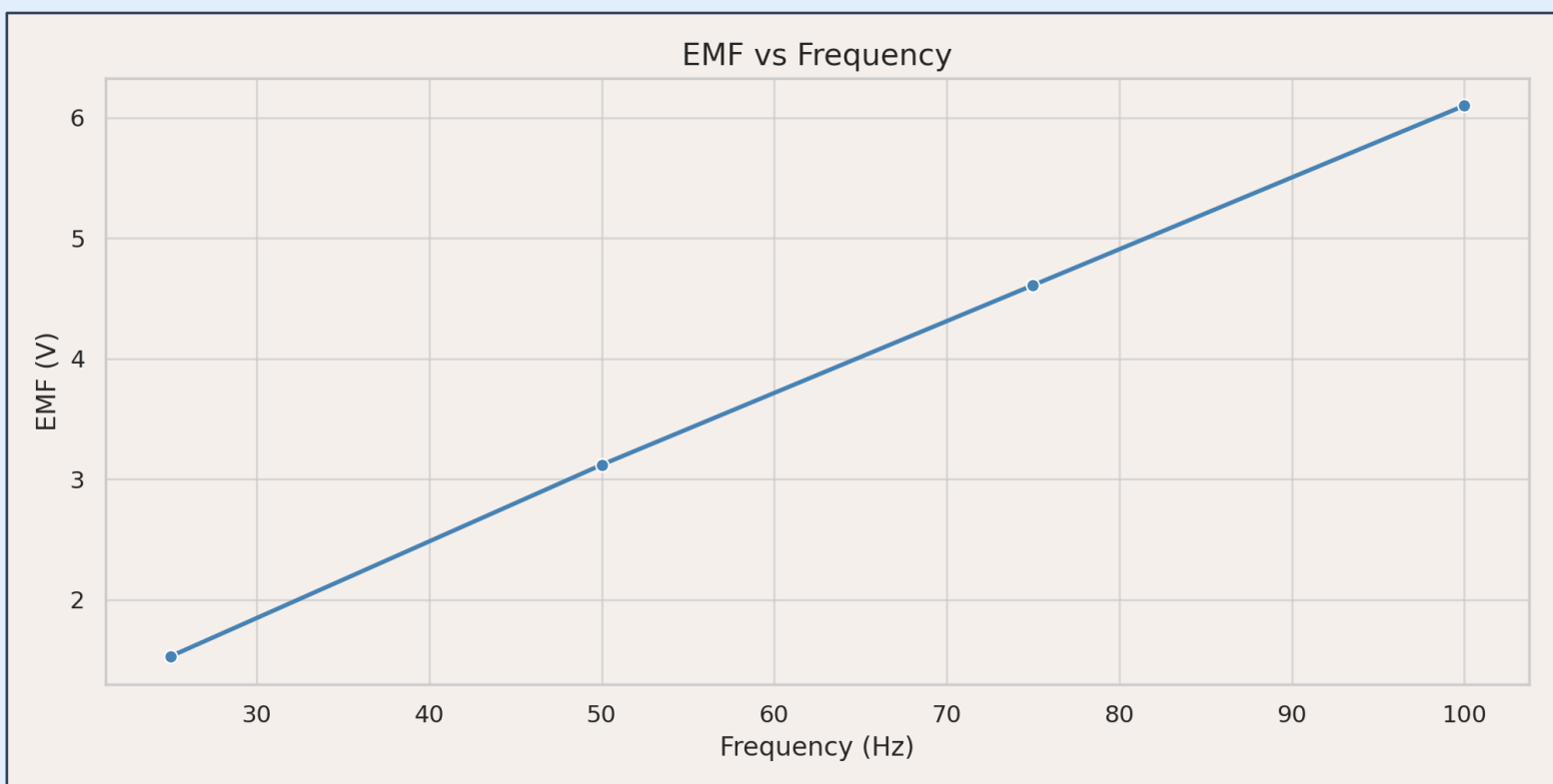
Table 1 Electromotive force and current frequency.

ε (mV)	I (mA)
3.15	5
6.1	10
9.24	15
12.11	20

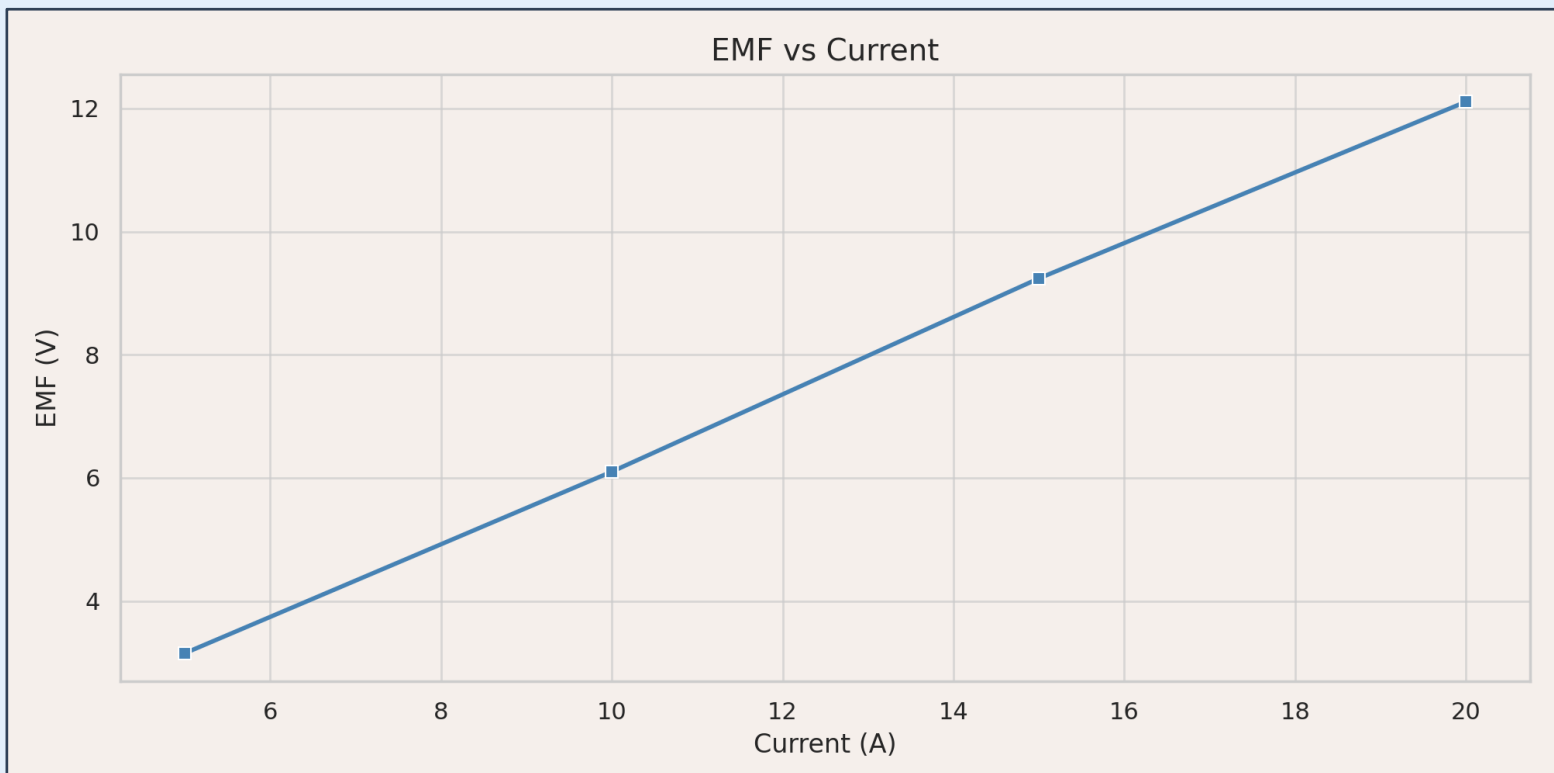
Table 2 Electromotive force and current intensity.

Equipment

a		AC Power supply (function generator)
b		Primary coil
c		Secondary coil
d		2 digital multimeters
e		Connecting wires



graph 1 for table1 Electromotive force and current frequency



Graph 2 for table 2 Electromotive force and current intensity.

Discussion

The relationship shown in both graphs is direct. In the first graph (EMF vs Frequency), increasing the frequency leads to a higher EMF because it increases the rate of change of magnetic flux. In the second graph (EMF vs Current), a higher current generates a stronger magnetic field, which results in a greater induced EMF when the field changes.

References

- [1] <https://byjus.com/magnetic-induction-formula/>
- [2] Halliday, D., Resnick, R., & Walker, J. (2013). Fundamentals of physics (10th ed.). Wiley. (Chapter 30 – Electromagnetic Induction). ISBN: 978-1118230718
- [3] Halliday, D., Resnick, R., & Walker, J. (2013). Fundamentals of Physics (10th ed.). Wiley.

Modelling the Experiment using Python

✦ A portion of Data tables Input code

```
import matplotlib.pyplot as plt
import pandas as pd
# Data for the tables
data1 = {
    'ε (mV)': [1.53, 3.12, 4.61, 6.1],
    'f (Hz)': [25, 50, 75, 100]
}
data2 = {
    'I (mA)': [5, 10, 15, 20],
    'ε (mV)': [3.15, 6.1, 9.24, 12.0]
}
# Create dataframes
df1 = pd.DataFrame(data1)
df2 = pd.DataFrame(data2)
# Set up the figure and axes
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
# Add a main title with blue color (note: #ff0000 is red, not blue)
fig.suptitle('Electromotive Force Measurements', fontsize=34,
y=1.1, color='#2F4156')
```

✦ Data of tables Output results

Electromotive Force Measurements			
1. Electromotive force and current frequency		2. Electromotive force and current intensity	
ε (mV)	f (Hz)	I (mA)	ε (mV)
1.53	25.0	5.0	3.15
3.12	50.0	10.0	6.1
4.61	75.0	15.0	9.24
6.1	100.0	20.0	12.0

✦ Discussion of Results Input code

```
from rich.console import Console
from rich.text import Text
from rich.style import Style
# Initialize console
console = Console()
# Color definitions
HEADER_LABEL_COLOR = "#2F4156" # Navy
CONTENT_COLOR = "#567C8D" # Teal
# Create styles — All bold
styles = {
    "header": Style(color=HEADER_LABEL_COLOR, bold=True),
    "label": Style(color=HEADER_LABEL_COLOR, bold=True),
    "content": Style(color=CONTENT_COLOR, bold=True),
    "item": Style(color=CONTENT_COLOR, bold=True)
}
def print_formatted_output():
    """Print the formatted output using bold text and custom colors"""
    question = (
        "Determine the type of relationship between the electric EMF generated "
        "in the secondary coil and the frequency of the current passing in the "
        "primary coil."
    )
    answer = "Direct relationship between electric EMF and frequency."
    console.print(Text("==== Discussion & Analysis of Results ===",
    style=styles["header"]))
    console.print(
        Text("Question: ", style=styles["label"]) +
        Text(question, style=styles["content"])
    )
    console.print(
        Text("Answer: ", style=styles["label"]) +
        Text(answer, style=styles["content"])
    )
# Run the function
print_formatted_output()
```

✦ Discussion of Results output results

```
==== Discussion & Analysis of Results ====
Question: Determine the type of relationship between the electric EMF generated in the secondary coil and the frequency of the current passing in the primary coil.
Answer: Direct relationship between electric EMF and frequency.
```

Electromagnetic Induction Simulation

✦ self learning part ✦

This project presents an interactive simulation of **electromagnetic induction** using the Python library **Pygame**. The simulation visually demonstrates how the movement of a magnet near a coil induces an electric current, which in turn lights up a bulb. The system includes key visual elements such as a multi-turn coil, a draggable magnet, animated magnetic field lines, and a dynamic graph displaying the induced current over time. The simulation responds in real time to user input, making the physical concept more engaging and easier to understand. The project utilizes **Pygame** for real-time graphics and interaction, **NumPy** for efficient data handling and current visualization, and **Math** for precise geometric calculations. This simulation bridges the gap between physics theory and programming practice, offering a hands-on learning experience.

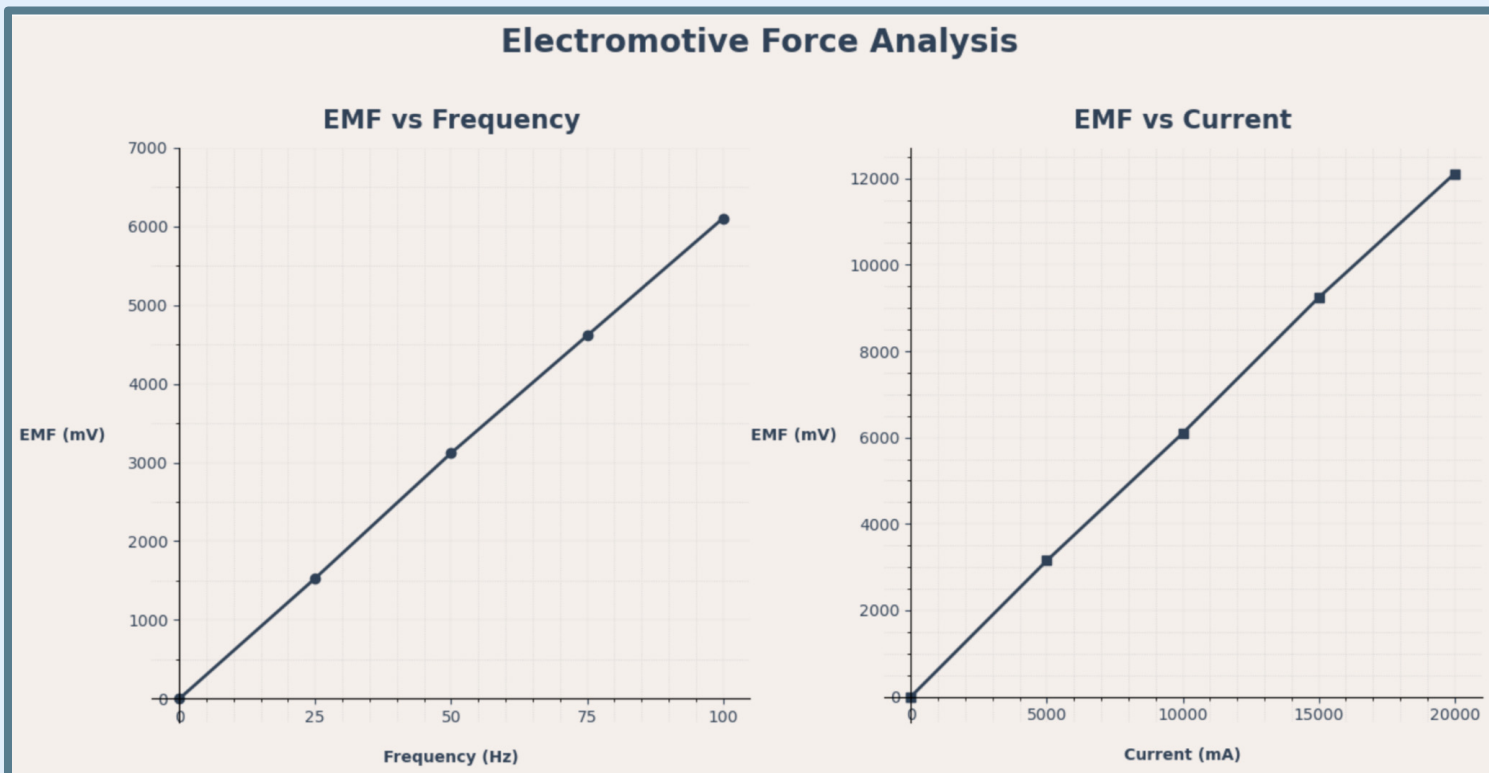
Scan here



✦ A portion of Analysis Results Input

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
# Data (pre-converted to mV and mA)
data = {
    'Frequency': {'Hz': [25, 50, 75, 100]}, 'mV': [1530, 3120, 4610, 6100]},
    'Current': {'mA': [5000, 10000, 15000, 20000]}, 'mV': [3150, 6100, 9240, 12110]}
}
# Updated color scheme
background_color = '#F5EFEB' # Beige background
font_color = '#2F4156' # Navy font
# Create figure with specified background color
fig = plt.figure(figsize=(13, 6.5), facecolor=background_color)
# Add main title header
plt.suptitle('Electromotive Force Analysis',
    fontsize=20,
    fontweight='bold',
    color=font_color,
    y=1.02)
```

✦ Analysis Results Output results



✦ Accuracy Input code

```
def calculate_fixed_device_accuracy(smallest_freq_unit,
smallest_current_unit, smallest_voltage_unit):
    """Calculates the 'device accuracy' based on provided smallest unit values for each device."""
    return {
        'frequency_accuracy': smallest_freq_unit / 2,
        'current_accuracy': smallest_current_unit / 2,
        'voltage_accuracy': smallest_voltage_unit / 2
    }
# 24-bit color escape formatter
def rgb_escape(hex_color: str, bold=True):
    hex_color = hex_color.lstrip('#')
    r, g, b = int(hex_color[:2], 16), int(hex_color[2:4], 16), int(hex_color[4:], 16)
    return f"\033[1;38;2;{r};{g};{b}m" if bold else f"\033[38;2;{r};{g};{b}m"
# Constants for formatting
RESET = "\033[0m"
NAVY_BOLD = rgb_escape("#2F4156", bold=True)
TEAL_BOLD = rgb_escape("#567C8D", bold=True)
if __name__ == "__main__":
    smallest_freq = 0.001 # Hz
    smallest_current = 0.1 # mA
    smallest_voltage = 0.01 # mV
    device_accuracy = calculate_fixed_device_accuracy(smallest_freq, smallest_current, smallest_voltage)
    # Output with exact colors
    print(f"{NAVY_BOLD}Estimated Device Accuracy (based on provided smallest unit values){RESET}")
    print(f"{TEAL_BOLD}Frequency Device Accuracy: ± {device_accuracy['frequency_accuracy']:.5f} Hz{RESET}")
    print(f"{TEAL_BOLD}Current Device Accuracy: ± {device_accuracy['current_accuracy']:.2f} mA{RESET}")
    print(f"{TEAL_BOLD}Voltage Device Accuracy: ± {device_accuracy['voltage_accuracy']:.4f} mV{RESET}")
```

✦ Accuracy Output results

```
Estimated Device Accuracy (based on provided smallest unit values):
Frequency Device Accuracy: ± 0.00050 Hz
Current Device Accuracy: ± 0.05 mA
Voltage Device Accuracy: ± 0.0050 mV
```

✦ A portion of the simulation input code

```
pip install pygame numpy
import pygame
import sys
import math
import numpy as np
# Initialize Pygame
pygame.init()
# Screen settings
WIDTH, HEIGHT = 1000, 650
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Electromagnetic Induction Simulation")
# Colors
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (200, 30, 30)
BLUE = (30, 30, 200)
YELLOW = (255, 255, 150)
GRAY = (150, 150, 150)
DARK_GRAY = (80, 80, 80)
GREEN = (50, 200, 50)
```

✦ simulation output results

